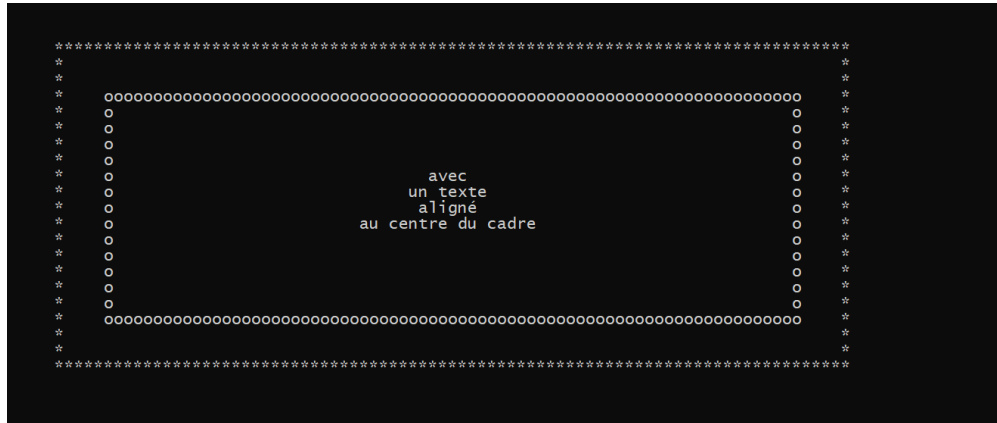


Exercice complémentaire sur les fonctions

Le but de cet exercice est de vous aider à mieux comprendre le principe des fonctions, des paramètres qui leur sont passés et des résultats qu'elles renvoient, en se concentrant sur **l'utilisation** de ces fonctions.

Nous allons écrire une série de fonctions permettant de dessiner la figure suivante :



Aucune des fonctions n'est particulièrement compliquée (la plus longue fait sept lignes ; les plus courtes n'en font qu'une) et il n'est pas nécessaire de traiter les cas particuliers où les données seraient incorrectes : vous devez surtout vous concentrer sur :

- Les paramètres dont la fonction a besoin.
- Le résultat qu'elle doit renvoyer (certaines ne renvoient rien).
- Surtout, la manière dont on peut utiliser les premières fonctions pour écrire facilement les suivantes.

Après avoir fait l'exercice (et, comme d'habitude, surtout pas avant !), allez voir la solution proposée. Faites-le même si votre programme donne bien le résultat attendu : la question n'est pas de savoir si vous avez réalisé le bon dessin, mais si vous avez correctement tiré parti des fonctions.

1. Ecrire une fonction `DessinerLigneHorizontale()`. Cette fonction reçoit en paramètres les coordonnées¹ horizontale et verticale du début de la ligne, ainsi que sa longueur et le caractère à écrire, et trace la ligne horizontale correspondante.
2. Ecrire une fonction `DessinerLigneVerticale()` qui se comporte de la même manière, mais dessine une ligne verticale.

Pour la fonction dessinant un cadre, nous ne parlerons pas en termes de longueur et largeur, mais en indiquant où doivent être les bords gauche, droit, haut et bas du cadre. Nous aurons donc besoin d'une fonction permettant de calculer la hauteur et largeur nécessaires.

3. Ecrire une fonction `Distance()`, qui reçoit une valeur de début et une de fin et renvoie la distance entre ces valeurs.

Remarques :

- Cette fonction nous permettra de déterminer combien de caractères il faut écrire pour tracer une ligne (verticale ou horizontale, mais pas oblique) entre deux positions.
- Il n'y a pas besoin de coordonnées complètes : si l'on veut savoir combien de caractères tracer entre les points (4, 8) et (10, 8), par exemple, il suffira de demander la distance entre 4 et 10.
- La distance est égale à $(fin - debut + 1)$, pour englober le point de début et celui de fin.

4. Ecrire une fonction `DessinerCadre()`. Cette fonction reçoit les coordonnées du coin supérieur gauche et du coin inférieur droit (les bords doivent être inclus dans le cadre), ainsi que le caractère à écrire, et dessine le cadre correspondant.

Remarque :

- Un cadre se compose de deux lignes horizontales et deux lignes verticales : il est évidemment utile de faire appel aux fonctions ci-dessus ! `Distance()` permet de calculer quelles longueurs passer à ces fonctions.

5. Ecrire une fonction `Milieu()` qui reçoit deux nombres et renvoie la valeur du milieu entre ces nombres, arrondie vers le bas si nécessaire. Il faut réfléchir un peu pour voir comment calculer ce milieu, mais ce n'est pas bien compliqué.

¹ Pour toutes les coordonnées, nous compterons de gauche à droite, de haut en bas, à partir de 0, comme pour la fonction `Console.SetCursorPosition()`.

6. Ecrire une fonction `EcrireCentre()` qui reçoit les coordonnées des bords gauche et droit d'un cadre, ainsi qu'une position verticale et un texte, et écrit ce texte à la position verticale voulue, en le centrant horizontalement par rapport au cadre.

Remarque :

- *Pour centrer le texte, il faut se placer au milieu, moins la moitié de la longueur du texte.*
- *Pour rappel, si `MonTexte` est de type `string`, on obtient sa longueur via `MonTexte.Length()`.*

7. Ecrivez dans `Main()` le code réalisant la figure. Aucune boucle ou autre structure à écrire ici : si vos fonctions sont bien faites, il y en a pour 7 lignes.

Pour les curieux : à propos de la documentation

Les commentaires de la solution ont été écrits selon les principes permettant la génération automatique de documentation. Ceci sort du cadre direct du cours, mais, si vous désirez vous y essayer, le principe est assez simple (voir par exemple <https://docs.microsoft.com/en-us/dotnet/csharp/codedoc> pour plus de détails):

- Avant chaque fonction, il faut décrire son comportement via des commentaires `///` et des balises XML permettant au compilateur de reconnaître les éléments de la description, par exemple `<summary>`, `<returns>`, `<remarks>`, ...
- Dans les propriétés du projet, section « build », il faut cocher la case « XML documentation file ». Un fichier XML rassemblant toute la doc est ainsi créé à la compilation.
- Des outils existent qui permettent de générer automatiquement des pages html et/ou un pdf de documentation. Par exemple, la doc que je vous transmets a été générée via Doxygen (<http://www.doxygen.nl/>).
- Notez que ceci génère également l'aide contextuelle, c'est-à-dire les messages flottants apparaissant sous Visual Studio lorsque vous tapez le nom d'une fonction.